

OBJECT-ORIENTED PROGRAMMING

Complete Interview Preparation Guide — A to Z

Java (Primary) · C++ (Comparisons & Additions)

On-Campus Fresher SDE Interviews

Covers: Java Basics · OOP Pillars · Constructors · Inheritance · Polymorphism · Abstraction · Interfaces · Encapsulation · Static · Final · Exceptions · Multithreading · Collections · Java 8 · Generics · Memory Management · Design Patterns · C++ Specifics

SECTION 1: Java Basics

1.1 Data Types

Q: What are the 8 primitive data types in Java?

Answer:

Type	Description	Range	Example
byte	8-bit signed integer	-128 to 127	byte b = 100;
short	16-bit signed integer	-32,768 to 32,767	short s = 1000;
int	32-bit signed integer	~±2.1 billion	int x = 42;
long	64-bit signed integer	~±9.2×10 ¹⁸	long l = 100L;
float	32-bit IEEE 754 float	~6-7 decimal digits	float f = 3.14f;
double	64-bit IEEE 754 float	~15-16 decimal digits	double d = 3.14;
char	16-bit Unicode character	0 to 65535	char c = 'A';
boolean	true/false	true or false	boolean b = true;

C++ C++ has same primitives + unsigned variants (unsigned int, unsigned long). Java has NO unsigned types except char.

1.2 Type Casting & Promotion

Q: What is implicit (widening) vs explicit (narrowing) casting?

Answer:

Widening (implicit): `byte` → `short` → `int` → `long` → `float` → `double`. No data loss. Automatic.

Narrowing (explicit): `double` → `float` → `long` → `int` → `short` → `byte`. Possible data loss. Must cast manually.

```
int i = 100;
long l = i;           // widening - automatic
double d = l;         // widening - automatic

double x = 9.99;
int y = (int) x;      // narrowing - explicit cast, y = 9 (truncated)

// Type promotion in expressions
byte a = 10, b = 20;
// byte c = a + b; // ERROR! a+b promotes to int
byte c = (byte) (a + b); // explicit narrowing needed
```

1.3 Strings, StringBuilder, StringBuffer

Q: What is the difference between String, StringBuilder, and StringBuffer?

Answer:

Class	Key Property	When to Use
String	Immutable. Stored in String Pool. Thread-safe by design.	String s = "hello"; s += "world"; // creates NEW object
StringBuilder	Mutable. NOT thread-safe. Faster than StringBuffer.	sb.append("hi").append(123); // in-place modification
StringBuffer	Mutable. Thread-safe (synchronised methods). Slower.	Use in multi-threaded code requiring shared buffer

```
// String immutability - IMPORTANT interview point
String s1 = "hello";
String s2 = "hello";
System.out.println(s1 == s2);      // true - same pool reference
String s3 = new String("hello");
System.out.println(s1 == s3);      // false - different heap object
System.out.println(s1.equals(s3)); // true - same content

// StringBuilder for concatenation in loops - O(n) vs O(n^2)
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) sb.append(i);
String result = sb.toString();
```

1.4 Arrays

Q: How are arrays different in Java vs C++?

Answer:

```
// Java - array is an OBJECT on the heap
int[] arr = new int[5];           // heap allocation
int[] arr2 = {1, 2, 3, 4, 5};     // array literal
System.out.println(arr.length);  // built-in length property
// ArrayIndexOutOfBoundsException on out-of-range access

// 2D array
int[][] matrix = new int[3][4];
int[][] jagged = new int[3][];    // rows can have different lengths
```

C++ C++ arrays: `int arr[5];` // stack allocated. No bounds checking. `arr` is just a pointer. Use `std::vector<int>` for dynamic size (equivalent to Java `ArrayList<Integer>`).

SECTION 2: OOP — The Four Pillars

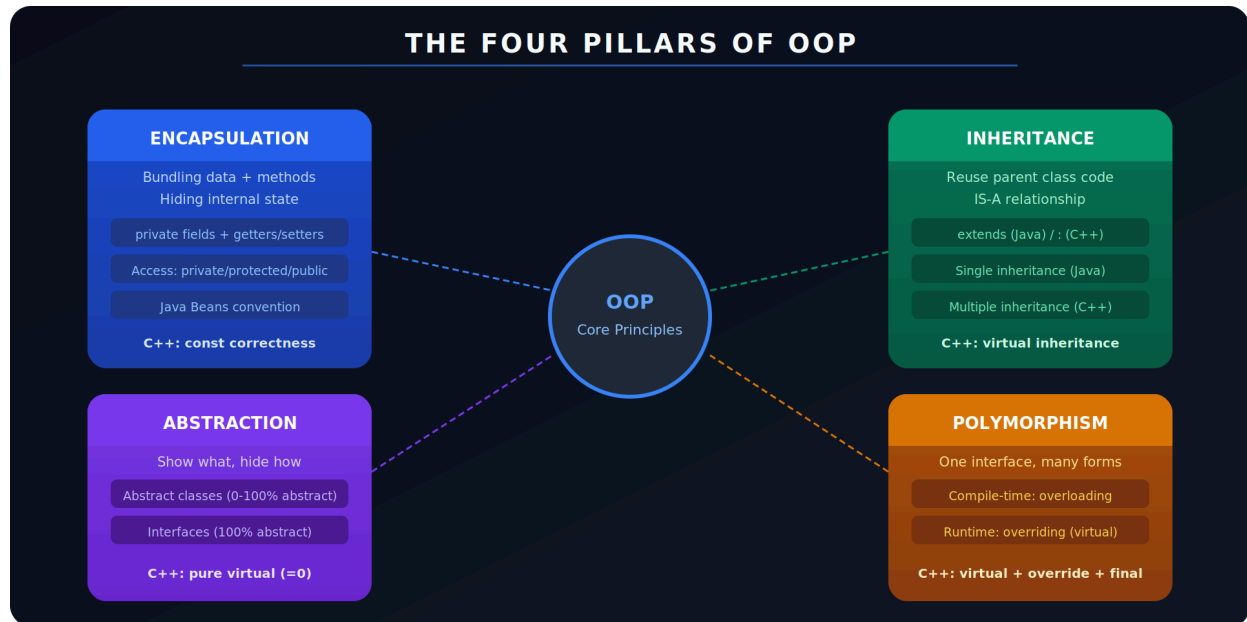


Figure 1: The Four Pillars of OOP — Encapsulation, Abstraction, Inheritance, Polymorphism

2.1 Encapsulation

Q: What is encapsulation? Why is it important?

Answer:

Definition: Bundling data (fields) and methods that operate on that data within a single unit (class), and restricting direct access to some components via access modifiers.

Benefits: (1) Data hiding — prevent invalid states. (2) Flexibility — change internal implementation without breaking external code. (3) Maintainability — clear public API.

```
public class BankAccount {  
    private double balance;    // private — cannot be accessed directly  
    private String accountId;  
  
    public double getBalance() {    // controlled read access  
        return balance;  
    }  
  
    public void deposit(double amount) {    // controlled write with validation  
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");  
        balance += amount;  
    }  
    // No setter for accountId — immutable after construction  
}
```

C++ C++: same access modifiers (private, protected, public). Add const correctness: const methods cannot modify object state: `double getBalance() const { return balance; }`

2.2 Abstraction

Q: What is abstraction? How is it achieved in Java?

Answer:

Definition: Hiding implementation details and exposing only the necessary interface. Show WHAT an object does, not HOW it does it.

Two mechanisms in Java:

- **Abstract classes:** 0% to 99% abstract. Can have fields, constructors, concrete methods. Use when subclasses share common state/behaviour.
- **Interfaces:** 100% abstract (before Java 8). Define a contract. A class can implement multiple interfaces — Java's solution to multiple inheritance.

```
// Abstract class
abstract class Shape {
    protected String colour;           // shared state
    Shape(String c) { colour = c; }    // constructor
    abstract double area();             // abstract method — must override
    void printColour() { System.out.println(colour); } // concrete method
}

// Interface
interface Drawable {
    void draw();                       // implicitly public abstract
    default void erase() { System.out.println("Erasing"); } // Java 8 default
    static Drawable noop() { return () -> {}; } // Java 8 static
}

class Circle extends Shape implements Drawable {
    private double radius;
    Circle(double r) { super("red"); radius = r; }
    @Override public double area() { return Math.PI * radius * radius; }
    @Override public void draw() { System.out.println("Drawing circle"); }
}
```

C++ C++ pure virtual = abstract: virtual double area() = 0; A class with any pure virtual method is abstract. No interface keyword — use abstract class with all pure virtuals.

2.3 Inheritance

Q: Explain inheritance with types. Why does Java not support multiple inheritance?

Answer:

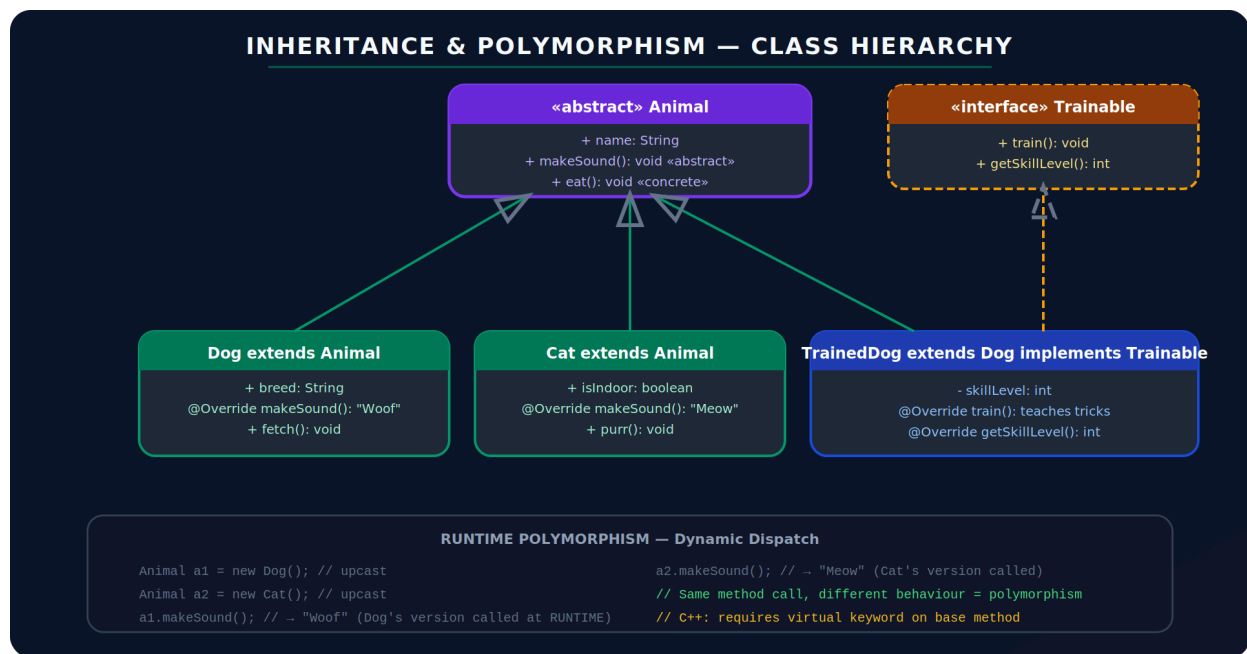


Figure 2: Inheritance Hierarchy — Animal → Dog/Cat → TrainedDog implements Trainable

Types of inheritance:

Type	Description	Example
Single	One parent, one child	Dog extends Animal
Multi-level	Chain: A → B → C	TrainedDog extends Dog extends Animal
Hierarchical	One parent, multiple children	Dog, Cat both extend Animal
Multiple	Two+ parents — NOT in Java	Java uses interfaces instead
Hybrid	Mix of above — NOT in Java directly	Via interfaces

Why no multiple class inheritance in Java? The Diamond Problem. If class C extends both A and B, and both A and B define method foo(), which version does C inherit? Ambiguity. Java solves this by allowing multiple **interface** implementation (since interfaces have no state, the conflict is resolved by the implementing class).

C++ C++ allows multiple inheritance: class C : public A, public B {}. Diamond problem solved via virtual inheritance: class A : virtual public Base {};

2.4 Polymorphism

Q: Explain compile-time vs runtime polymorphism with examples.

Answer:

Type	Mechanism	When Resolved	Example
Compile-time (Static)	Method Overloading	Resolved at compile time based on method signature	add(int,int) vs add(double,double)

Runtime (Dynamic)	Method Overriding	Resolved at runtime based on actual object type	Animal ref → Dog.makeSound()
-------------------	-------------------	---	------------------------------

```
// Compile-time polymorphism - Overloading
class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; } // different params
    int add(int a, int b, int c) { return a + b + c; } // different arity
}

// Runtime polymorphism - Overriding + Upcasting
Animal a = new Dog(); // reference type: Animal, object type: Dog
a.makeSound(); // calls Dog.makeSound() - decided at RUNTIME
// a.fetch(); // COMPILER ERROR - Animal ref doesn't know fetch()

// instanceof check before downcast
if (a instanceof Dog d) { // Java 16+ pattern matching
    d.fetch();
}
```

C++ C++: runtime polymorphism REQUIRES 'virtual' keyword on base method. Without virtual, base class method is called (static dispatch). Java methods are virtual by default.

SECTION 3: Constructors, Keywords & Class Members

3.1 Constructors & Overloading

Q: What are the constructor types? What is constructor chaining?

Answer:

```
class Person {
    private String name;
    private int age;

    // Default constructor (compiler adds if none defined)
    Person() { this("Unknown", 0); } // constructor chaining via this()

    // Parameterised constructor
    Person(String name, int age) {
        this.name = name; // 'this' disambiguates field vs param
        this.age = age;
    }

    // Copy constructor (Java - manual, no built-in)
    Person(Person other) {
        this.name = other.name;
        this.age = other.age;
    }
}

class Employee extends Person {
    private String company;
    Employee(String name, int age, String company) {
        super(name, age); // must be FIRST line - calls parent constructor
        this.company = company;
    }
}
```

C++ C++ has 3 special constructors: Default: `Person() {}`, Copy: `Person(const Person& other) {}`, Move: `Person(Person&& other) {}`. Also Destructor: `~Person() { // release resources }`. Rule of 3/5: if you define any of destructor/copy/move, define all.

3.2 this and super Keywords

Q: Explain 'this' and 'super' with all their uses.

Answer:

Usage	Purpose	Example
<code>this.field</code>	Refer to current object's field (vs local variable)	<code>this.name = name;</code>
<code>this()</code>	Call another constructor of same class (must be first line)	<code>this("default", 0);</code>

this (as ref)	Pass current object as argument	list.add(this);
super.method()	Call parent class method	super.toString()
super()	Call parent constructor (must be first line)	super(name, age);
super.field	Access parent field if shadowed	super.colour

3.3 Static Members & Blocks

Q: What is static? What is a static block?

Answer:

```
class Counter {
    private static int count = 0; // shared across ALL instances
    private int id;

    // Static block — runs ONCE when class is first loaded
    static {
        System.out.println("Counter class loaded");
        count = 10; // initialise static state
    }

    Counter() { id = ++count; }

    public static int getCount() { return count; } // static method
    // Cannot use 'this' or instance fields inside static methods
}

// Static methods called on CLASS, not instance
Counter.getCount();
Math.abs(-5); // Math.abs is static
```

C++ C++: static data member must be defined outside the class: `int Counter::count = 0;` Static methods same concept. No static blocks — use static member initialization or anonymous namespace.

3.4 Final Keyword

Q: What are the three uses of 'final' in Java?

Answer:

Usage	Effect	Example
final variable	Constant — cannot be reassigned after initialisation	final int MAX = 100;
final method	Cannot be overridden in subclasses	final void reset() {}
final class	Cannot be subclassed (e.g., String, Integer)	public final class String

C++ C++ equivalents: const variable, final method → override + final specifier in C++11: void makeSound() final; , sealed class → final class: class Dog final : public Animal {};

3.5 Access Modifiers

Q: Explain all four access modifiers in Java.

Answer:

Modifier	Scope	Notes
private	Class only	Can't access outside the class
default (no keyword)	Package	Accessible within same package only
protected	Package + subclasses	Subclasses in any package can access
public	Everywhere	No restriction

C++ C++ has private, protected, public (same meaning). Default is private for class, public for struct. No package concept — use namespaces. Friend keyword grants specific external class/function access.

SECTION 4: Exception Handling

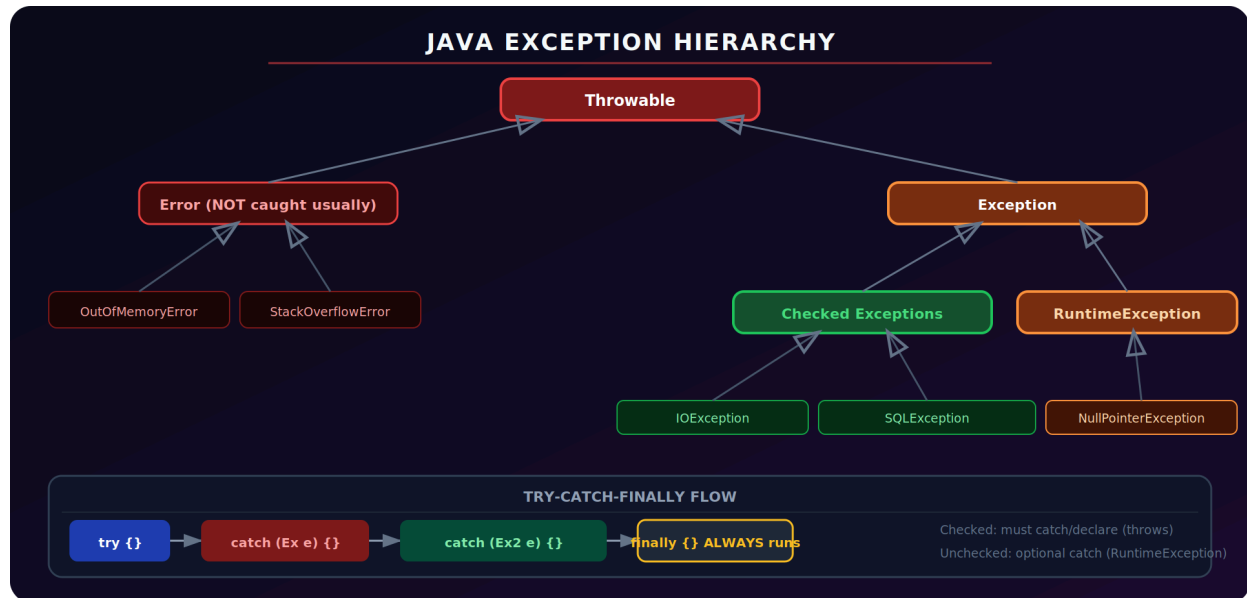


Figure 3: Java Exception Hierarchy — Throwable → Error / Exception → Checked / Unchecked

4.1 Checked vs Unchecked Exceptions

Q: What is the difference between checked and unchecked exceptions?

Answer:

Type	Hierarchy	Requirement	Examples
Checked	Extends Exception (not RuntimeException)	Must be caught or declared with 'throws'	IOException, SQLException, FileNotFoundException
Unchecked	Extends RuntimeException	Optional to catch	NullPointerException, ArrayIndexOutOfBoundsException, IllegalArgumentException
Error	Extends Error	Never catch — unrecoverable JVM problems	OutOfMemoryError, StackOverflowError

4.2 try-catch-finally & Custom Exceptions

Q: Show try-catch-finally and how to create a custom exception.

Answer:

```
// Custom checked exception
class InsufficientFundsException extends Exception {
    private double amount;
    InsufficientFundsException(double amount) {
```

```

        super("Insufficient funds: need " + amount);
        this.amount = amount;
    }
    public double getAmount() { return amount; }
}

// Using try-catch-finally
public void withdraw(double amount) throws InsufficientFundsException {
    if (amount > balance) throw new InsufficientFundsException(amount);
    balance -= amount;
}

try {
    account.withdraw(1000);
} catch (InsufficientFundsException e) {
    System.out.println(e.getMessage());
} catch (Exception e) {          // catch broader AFTER specific
    e.printStackTrace();
} finally {
    System.out.println("Always runs - close resources here");
}

// try-with-resources (Java 7+) - auto-closes Autocloseable
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    String line = br.readLine();
} // br.close() called automatically even if exception thrown

```

4.3 throw vs throws

Q: What is the difference between throw and throws?

Answer:

Keyword	Usage	Example
throw	Used inside method body to actually throw an exception object	throw new NullPointerException("Name is null");
throws	Used in method signature to declare that this method MAY throw	void read() throws IOException, SQLException

SECTION 5: Multithreading & Concurrency

5.1 Creating Threads

Q: What are the two ways to create a thread in Java? Which is preferred?

Answer:

```
// Way 1: Extend Thread class
class MyThread extends Thread {
    @Override
    public void run() { System.out.println("Thread: " + getName()); }
}
new MyThread().start(); // start() creates new thread; run() would run on current

// Way 2: Implement Runnable (PREFERRED - doesn't waste extends)
class MyTask implements Runnable {
    @Override
    public void run() { System.out.println("Task running"); }
}
new Thread(new MyTask()).start();

// Way 3: Lambda (Java 8+) - most concise
new Thread(() -> System.out.println("Lambda thread")).start();

// Way 4: Executor Framework (production preferred)
ExecutorService pool = Executors.newFixedThreadPool(4);
pool.submit(() -> System.out.println("Pool thread"));
pool.shutdown();
```

5.2 Thread Lifecycle

Q: Draw/explain the Thread lifecycle.

Answer:

State	Description	Trigger
NEW	Thread object created, start() not called yet	Thread t = new Thread(...)
RUNNABLE	start() called. Ready to run or running (JVM/OS decides)	t.start()
BLOCKED	Waiting to acquire a monitor lock (synchronised block)	Trying to enter synchronised
WAITING	Waiting indefinitely — needs notify() to wake	object.wait()
TIMED_WAITING	Waiting for specified time	Thread.sleep(1000), join(100)
TERMINATED	run() completed or exception thrown	Method returned

5.3 Synchronisation

Q: What is a race condition and how does synchronised fix it?

Answer:

```
// Race condition example - NOT thread-safe
class Counter { int count = 0; }
counter.count++; // read → increment → write: NOT atomic. Two threads can interleave.

// Fix 1: synchronized method
class SafeCounter {
    private int count = 0;
    public synchronized void increment() { count++; } // only 1 thread at a time
    public synchronized int get() { return count; }
}

// Fix 2: synchronized block (finer granularity)
public void increment() {
    synchronized(this) { // acquire lock on 'this'
        count++;
    }
}

// Fix 3: AtomicInteger (lock-free, fastest for single value)
AtomicInteger count = new AtomicInteger(0);
count.incrementAndGet(); // atomic CAS operation

// wait/notify - thread communication
synchronized(sharedObj) {
    while (!condition) sharedObj.wait(); // releases lock, waits
    // ... do work ...
    sharedObj.notifyAll(); // wake all waiting threads
}
```

SECTION 6: Java Collections Framework

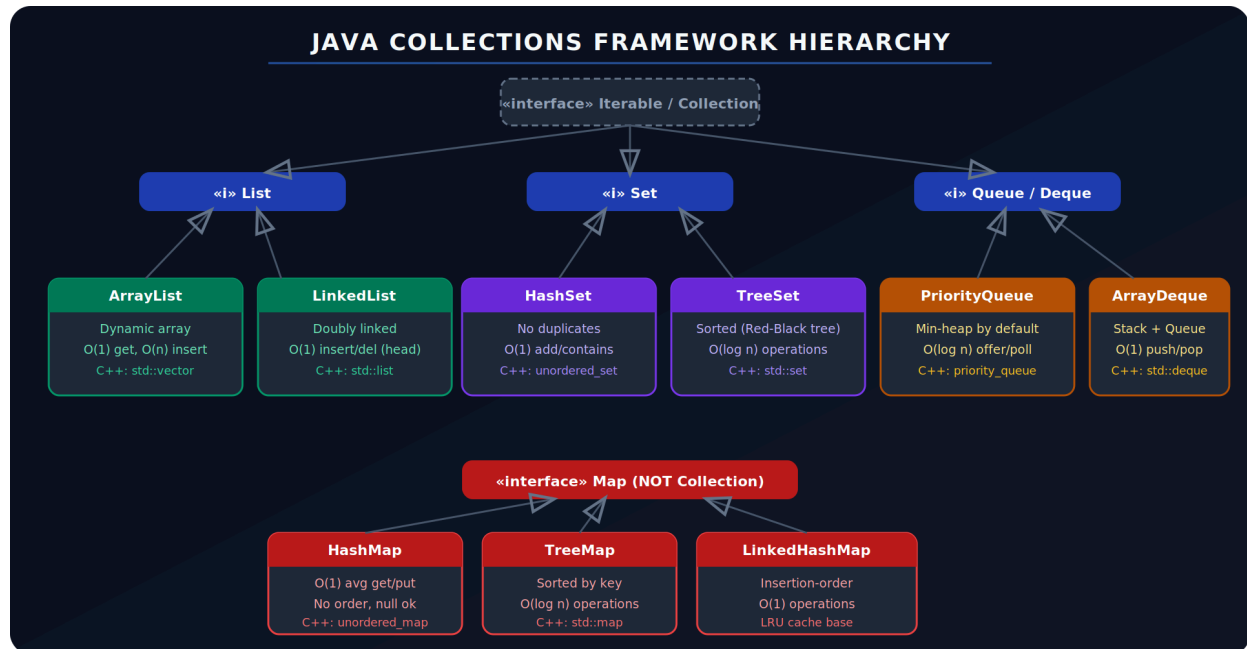


Figure 4: Java Collections Hierarchy — List, Set, Queue, Map

6.1 List — ArrayList vs LinkedList

Q: When would you use ArrayList vs LinkedList?

Answer:

Class	Characteristics	Use When
ArrayList	O(1) random access by index. O(n) insert/delete in middle. Backed by dynamic array. Cache-friendly.	Read-heavy operations, random access
LinkedList	O(n) random access. O(1) insert/delete at head/tail. Doubly-linked. Not cache-friendly.	Frequent insertions/deletions at ends, implementing Queue/Deque

C++ `std::vector` = ArrayList (dynamic array).
`std::list` = LinkedList (doubly linked).
`std::deque` = double-ended queue backed

by array
segments.

6.2 Set & Map

Q: Explain HashSet vs TreeSet, and HashMap vs TreeMap.

Answer:

Class	Properties	Notes
HashSet	O(1) add/contains/remove. No order. Backed by HashMap. Allows null.	HashSet<String> set = new HashSet<>();
TreeSet	O(log n). Sorted order (natural or Comparator). No null.	TreeSet<Integer> sorted = new TreeSet<>();
LinkedHashSet	O(1). Maintains insertion order.	Predictable iteration order
HashMap	O(1) avg. No key order. Allows 1 null key.	HashMap<String,Integer> map = new HashMap<>();
TreeMap	O(log n). Sorted by key. No null keys.	TreeMap for sorted iteration
LinkedHashMap	O(1). Insertion-order or access-order.	LRU cache implementation
ConcurrentHashMap	Thread-safe HashMap. Lock striping. O(1).	Multi-threaded access

6.3 Comparable vs Comparator

Q: What is the difference between Comparable and Comparator?

Answer:

```
// Comparable - natural ordering, implemented BY the class
class Student implements Comparable<Student> {
    String name; int gpa;
    @Override
    public int compareTo(Student other) {
        return Double.compare(this.gpa, other.gpa); // sort by GPA ascending
    }
}
Collections.sort(students); // uses compareTo

// Comparator - external ordering, passed separately
Comparator<Student> byName = (s1, s2) -> s1.name.compareTo(s2.name);
students.sort(byName);

// Chaining comparators
students.sort(Comparator.comparing((Student s) -> s.gpa)
    .thenComparing(s -> s.name));
```


SECTION 7: Java 8 Features

7.1 Lambda Expressions & Functional Interfaces

Q: What is a lambda expression and a functional interface?

Answer:

Functional Interface: An interface with exactly ONE abstract method (can have default/static). Annotated with `@FunctionalInterface`. Can be represented by a lambda.

```
@FunctionalInterface
interface MathOperation { int operate(int a, int b); }

// Traditional anonymous class
MathOperation add = new MathOperation() {
    public int operate(int a, int b) { return a + b; }
};

// Lambda - concise replacement
MathOperation add = (a, b) -> a + b;
MathOperation square = (a, b) -> { int r = a * a; return r; };

// Built-in functional interfaces
Predicate<String> isLong = s -> s.length() > 5;           // T → boolean
Function<String, Integer> len = String::length;           // T → R
Consumer<String> print = System.out::println;             // T → void
Supplier<List<String>> newList = ArrayList::new;           // () → T
BiFunction<Integer,Integer,Integer> mul = (a,b) -> a*b;
```

7.2 Stream API

Q: Show common Stream API operations.

Answer:

```
List<String> names = List.of("Alice", "Bob", "Charlie", "Anna", "Brian");

// filter → map → collect
List<String> result = names.stream()
    .filter(n -> n.startsWith("A"))           // intermediate: keep A names
    .map(String::toUpperCase)                 // intermediate: transform
    .sorted()                                // intermediate: sort
    .collect(Collectors.toList());            // terminal: materialise
// result: [ALICE, ANNA]

// reduce
int sum = IntStream.rangeClosed(1, 10).reduce(0, Integer::sum); // 55

// groupingBy
Map<Integer,List<String>> byLength = names.stream()
    .collect(Collectors.groupingBy(String::length));

// Optional - avoids NullPointerException
Optional<String> first = names.stream().filter(n -> n.startsWith("Z")).findFirst();
```

```
String val = first.getOrElse("Not found"); // safe default
first.ifPresent(System.out::println);    // only runs if present
```

SECTION 8: Java Memory Management

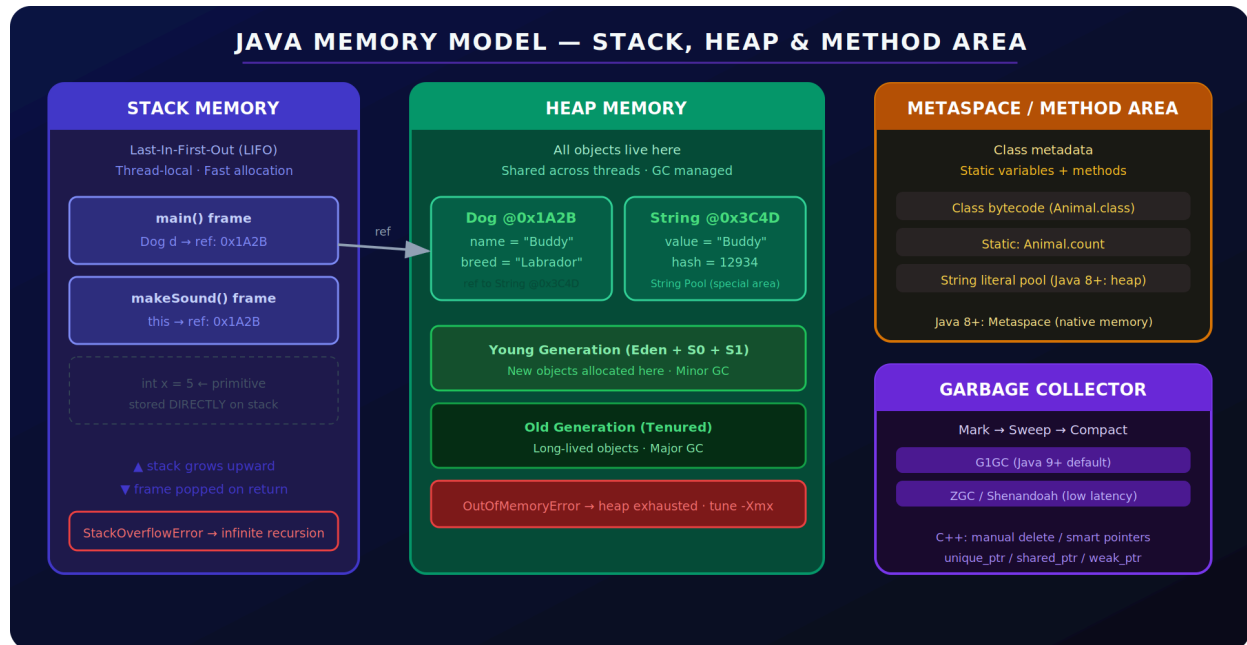


Figure 5: Java Memory Model — Stack, Heap, Metaspace, Garbage Collector

8.1 Stack vs Heap

Q: What is stored in Stack vs Heap memory?

Answer:

Area	Contents	Example
Stack	Primitive variables, object references (not objects), method call frames. Thread-local. LIFO. Fixed size per frame.	<code>int x = 5</code> (value on stack), <code>Dog d</code> (reference on stack)
Heap	All objects (<code>new Foo()</code>). Strings. Arrays. Shared across threads. GC managed.	<code>new Dog()</code> → object body on heap
Metaspace (Java 8+)	Class metadata, bytecode, static variables, string pool (Java 7+ moved to heap). Native memory.	Class definitions, static <code>int</code> count
Stack overflow	Infinite recursion exhausts stack frames	<code>StackOverflowError</code>
Heap overflow	Too many objects, GC cannot free enough	<code>OutOfMemoryError</code>

8.2 Garbage Collection

Q: How does the Java Garbage Collector work?

Answer:

Mark & Sweep: (1) Mark: GC roots (stack refs, static refs) trace all reachable objects. (2) Sweep: unmarked objects freed. (3) Compact: move live objects together to defragment heap.

Generational GC: Young Generation (Eden + S0 + S1): new objects here. Most die young (infant mortality). Minor GC is fast. Old Generation (Tenured): objects that survived multiple minor GCs. Major GC is slower.

Modern collectors:

- **G1GC (Java 9+ default):** Region-based. Predictable pause times. Good for large heaps.
- **ZGC:** Sub-millisecond pauses. Concurrent. For latency-critical applications.

```
// Make object eligible for GC
Dog d = new Dog("Rex");
d = null;           // reference removed - Dog eligible for GC
d = new Dog("Max"); // original Rex now unreachable

// finalize() - deprecated Java 9, removed Java 18
// Use Cleaner API or try-with-resources instead

// JVM flags for GC tuning
// -Xms512m  (initial heap size)
// -Xmx2g    (max heap size)
// -XX:+UseG1GC (use G1 collector)
```

C++ C++ has NO GC. Manual memory management: new allocates on heap, delete frees. Memory leak = forget to delete. Double-free = delete twice. Solution: RAII + smart pointers. unique_ptr auto-deletes when it goes out of scope. shared_ptr deletes when ref count hits 0. Destructor ~ClassName() called when object is destroyed.

SECTION 9: Advanced Topics

9.1 Generics & Wildcards

Q: What are generics and what is the difference between `<T>`, `<? extends T>`, `<? super T>`?

Answer:

```
// Generic class
class Box<T> {
    private T value;
    Box(T v) { value = v; }
    T get() { return value; }
}

Box<Integer> intBox = new Box<>(42);
Box<String> strBox = new Box<>("hello");

// Generic method
<T extends Comparable<T>> T max(T a, T b) { return a.compareTo(b) > 0 ? a : b; }

// Wildcards
// <? extends T> - Upper bounded: read-only (producer). You can read T.
void printAll(List<? extends Number> list) { for (Number n : list) System.out.println(n); }

// <? super T> - Lower bounded: write-only (consumer). You can add T.
void addNumbers(List<? super Integer> list) { list.add(42); }

// PECS rule: Producer Extends, Consumer Super
```

9.2 Inner Classes & Enums

Q: What are the types of inner classes?

Answer:

Type	Description	Example
Static nested class	Declared static inside outer. No outer instance needed. Like a top-level class in a namespace.	class Outer { static class Inner {} }
Inner (non-static)	Has reference to outer instance. Can access outer fields.	class Outer { class Inner { outer.field } }
Local class	Defined inside a method. Scoped to method.	void method() { class Local {} }
Anonymous class	Unnamed class defined and instantiated inline. Old way before lambdas.	Runnable r = new Runnable() { public void run(){} };

9.3 Annotations

Q: What are common Java annotations and what do they do?

Answer:

Annotation	Purpose	Note
@Override	Compiler checks that method actually overrides parent	Prevents typo bugs in method name
@Deprecated	Marks method/class as outdated	Generates compiler warning on use
@SuppressWarnings	Suppresses specific compiler warnings	@SuppressWarnings("unchecked")
@FunctionalInterface	Marks interface as having exactly one abstract method	Compile error if >1 abstract method
@SafeVarargs	Suppresses heap pollution warning for varargs	For generic varargs methods

9.4 Reflection API

Q: What is reflection? When would you use it?

Answer:

Reflection: The ability to inspect and modify class structure and behaviour at runtime — get class name, list methods/fields, invoke methods, create instances — without knowing the type at compile time.

```
Class<?> clazz = Class.forName("com.example.Dog");
Method[] methods = clazz.getDeclaredMethods();
Object dog = clazz.getDeclaredConstructor(String.class).newInstance("Rex");
Method makeSound = clazz.getMethod("makeSound");
makeSound.invoke(dog); // calls Dog.makeSound() at runtime
```

Use cases: Frameworks (Spring DI, JUnit, Hibernate ORM), serialisation libraries, plugin systems, debuggers. Avoid in normal application code — breaks encapsulation, slower, not type-safe.

SECTION 10: Design Patterns

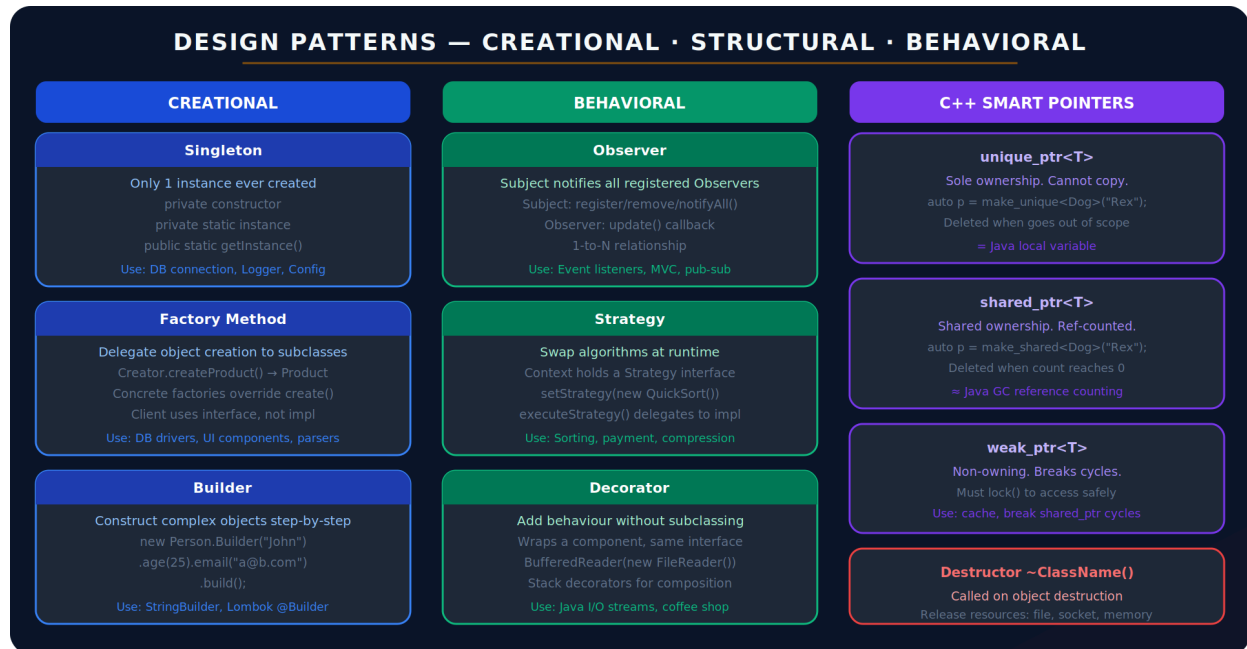


Figure 6: Design Patterns — Creational, Behavioral, C++ Smart Pointers & Destructor

10.1 Singleton

Q: Implement thread-safe Singleton in Java.

Answer:

```
// Double-checked locking - thread-safe + lazy init
public class DatabaseConnection {
    private static volatile DatabaseConnection instance; // volatile prevents reordering

    private DatabaseConnection() {} // private constructor

    public static DatabaseConnection getInstance() {
        if (instance == null) { // first check (no lock)
            synchronized (DatabaseConnection.class) {
                if (instance == null) { // second check (with lock)
                    instance = new DatabaseConnection();
                }
            }
        }
        return instance;
    }
}

// Better: Bill Pugh Singleton (class loading is thread-safe)
public class Singleton {
    private Singleton() {}
    private static class Holder { static final Singleton INSTANCE = new Singleton(); }
    public static Singleton getInstance() { return Holder.INSTANCE; }
}
```

10.2 Factory Pattern

Q: Implement the Factory pattern.

Answer:

```
interface Animal { void makeSound(); }
class Dog implements Animal { public void makeSound() { System.out.println("Woof"); } }
class Cat implements Animal { public void makeSound() { System.out.println("Meow"); } }

class AnimalFactory {
    public static Animal create(String type) {
        return switch (type.toLowerCase()) {
            case "dog" -> new Dog();
            case "cat" -> new Cat();
            default -> throw new IllegalArgumentException("Unknown: " + type);
        };
    }
}

Animal a = AnimalFactory.create("dog");
a.makeSound(); // Woof - client doesn't know it's a Dog
```

10.3 Observer Pattern

Q: Implement Observer pattern.

Answer:

```
interface Observer { void update(String event); }

class EventManager {
    private List<Observer> listeners = new ArrayList<>();
    public void subscribe(Observer o) { listeners.add(o); }
    public void unsubscribe(Observer o) { listeners.remove(o); }
    public void notifyAll(String event) { listeners.forEach(o -> o.update(event)); }
}

class Logger implements Observer {
    public void update(String event) { System.out.println("LOG: " + event); }
}

class EmailNotifier implements Observer {
    public void update(String event) { System.out.println("EMAIL: " + event); }
}

EventManager em = new EventManager();
em.subscribe(new Logger());
em.subscribe(new EmailNotifier());
em.notifyAll("User registered"); // both get notified
```

10.4 Strategy Pattern

Q: Implement Strategy pattern.

Answer:

```
interface SortStrategy { void sort(int[] arr); }
class BubbleSort implements SortStrategy {
    public void sort(int[] arr) { /* bubble sort logic */ }
}
class QuickSort implements SortStrategy {
    public void sort(int[] arr) { /* quicksort logic */ }
}

class Sorter {
    private SortStrategy strategy;
    public void setStrategy(SortStrategy s) { strategy = s; }
    public void sort(int[] arr) { strategy.sort(arr); } // delegate
}

Sorter sorter = new Sorter();
sorter.setStrategy(new QuickSort()); // swap algorithm at runtime
sorter.sort(data);
```

SECTION 11: C++ Specific Concepts

11.1 Pointers & References

Q: Explain pointers vs references in C++. Compare to Java references.

Answer:

```
// C++ Pointer - stores memory address, can be null, can be reassigned
int x = 10;
int* ptr = &x;      // ptr stores address of x
*ptr = 20;          // dereference: x is now 20
ptr = nullptr;      // can be null

// C++ Reference - alias for existing variable, cannot be null, cannot be reassigned
int& ref = x;        // ref IS x (same memory location)
ref = 30;            // x is now 30
// int& r2;          // ERROR - must initialise reference

// Java reference - like a C++ pointer but managed, no arithmetic, no null deref crash
Dog d = new Dog();  // d is a reference (like C++ Dog* d = new Dog();)
d = null;           // like ptr = nullptr; but no UB - NullPointerException on deref
```

11.2 Destructors & RAII

Q: What is a destructor? What is RAII?

Answer:

Destructor: Called automatically when an object goes out of scope (stack) or when delete is called (heap). Used to release resources — file handles, memory, network connections.

RAII (Resource Acquisition Is Initialisation): Resources are acquired in constructor and released in destructor. Guarantees cleanup even if exception thrown — deterministic unlike Java's finalizer/GC.

```
class FileHandler {
    FILE* file;
public:
    FileHandler(const char* path) {
        file = fopen(path, "r"); // acquire resource in constructor
        if (!file) throw std::runtime_error("Cannot open file");
    }
    ~FileHandler() {              // destructor - called automatically
        if (file) fclose(file); // release resource
        std::cout << "File closed\n";
    }
    // ... read methods ...
};

void process() {
    FileHandler fh("data.txt"); // constructor: file opened
    // ... use file ...
    // If exception thrown here, destructor STILL called
} // destructor called here: file closed automatically
```

11.3 Virtual Functions & Vtable

Q: How does C++ implement runtime polymorphism? What is a vtable?

Answer:

virtual keyword: Required in C++ for runtime polymorphism. Without virtual, base class method is called (static dispatch). Java methods are virtual by default.

Vtable (Virtual dispatch table): Compiler creates a table of function pointers for each class with virtual methods. Each object has a vptr pointing to its class's vtable. At runtime, method call is resolved by looking up the vtable — O(1) overhead.

```
class Animal {
public:
    virtual void makeSound() { cout << "..."; } // virtual
    virtual ~Animal() {} // ALWAYS make destructor virtual!
};

class Dog : public Animal {
public:
    void makeSound() override { cout << "Woof"; } // override specifier (C++11)
    // override = Java @Override - compile error if not actually overriding
};

Animal* a = new Dog();
a->makeSound(); // "Woof" - vtable lookup finds Dog::makeSound
delete a; // calls Dog destructor THEN Animal destructor (virtual dtor!)

// Without virtual on makeSound:
// a->makeSound() would call Animal::makeSound() - static dispatch
```

SECTION 12: Top Interview Questions & Tricky Concepts

12.1 Classic Tricky Questions

Q: What is the difference between == and .equals() in Java?

Answer:

==: Compares references (memory addresses) for objects. Compares values for primitives.

.equals(): Compares content. Object.equals() defaults to ==, but String, Integer, etc. override it to compare values.

```
String s1 = "hello"; String s2 = "hello";
s1 == s2           // true (same String pool object)
s1 = new String("hello");
s1 == s2           // false (different heap objects)
s1.equals(s2)      // true (same content)
Integer a = 127; Integer b = 127; a == b // true (cache -128 to 127)
Integer c = 128; Integer d = 128; c == d // false (outside cache)
```

Q: Can we override a static method? What is method hiding?

Answer:

No, static methods cannot be overridden. They belong to the class, not the instance. If a subclass defines a static method with the same signature, it **hides** the parent's method — it does NOT override it. No dynamic dispatch.

```
class Parent { static void greet() { System.out.println("Parent"); } }
class Child extends Parent { static void greet() { System.out.println("Child"); } }

Parent p = new Child();
p.greet(); // 'Parent' - static binding, no vtable
Child c = new Child();
c.greet(); // 'Child'
```

Q: What is the difference between abstract class and interface? When to use which?

Answer:

Property	Abstract Class	Interface
Can have constructor	Yes	No
Can have instance fields	Yes	No (only static final constants)
Can have concrete methods	Yes	Yes (default/static, Java 8+)
Multiple inheritance	No (single extends)	Yes (multiple implements)
Default access of methods	Can vary	public abstract

Use when	Shared code + state among related classes	Define contract for unrelated classes
Example	Vehicle (shared fields: speed, fuel)	Serializable, Comparable, Runnable

Q: What happens if you don't call `super()` in a subclass constructor?

Answer:

Java compiler automatically inserts `super()` as the first line if the parent has a no-arg constructor. If the parent has NO no-arg constructor, it is a compile error — you **MUST** explicitly call the appropriate `super()` constructor.

Q: What is covariant return type?

Answer:

In Java 5+, an overriding method can return a subtype of the parent's return type. Example: parent returns `Animal`, override can return `Dog` (since `Dog IS-A Animal`). This is covariant return type.

Q: Explain 'is-a' vs 'has-a' relationships.

Answer:

Relationship	Mechanism	Example
is-a (Inheritance)	Use extends/implements	Dog IS-A Animal → class Dog extends Animal
has-a (Composition)	Hold a reference as a field	Car HAS-A Engine → class Car { Engine engine; }
Prefer composition over inheritance	More flexible, avoids tight coupling	Strategy pattern uses composition

12.2 C++ vs Java Quick Comparison

Feature	Java	C++
Memory management	GC (automatic)	Manual (new/delete) or smart pointers
Multiple inheritance	Via interfaces only	Full class + interface
Runtime polymorphism	Default (all methods virtual)	Requires 'virtual' keyword
Pointers	No (managed references)	Yes (raw + smart pointers)
Destructor	<code>finalize()</code> (deprecated), GC	<code>~ClassName()</code> — deterministic RAII

String	Immutable String class	std::string (mutable)
Default access	package-private	private for class, public for struct
Operator overloading	No	Yes
Templates/Generics	Generics (type erasure at runtime)	Templates (compile-time code gen)
Exception specification	Checked + unchecked	noexcept specifier

SECTION 13: Quick Reference Cheat Sheet

13.1 OOP Keywords — Java

Keyword	Meaning
extends	Inherit from a class (single only)
implements	Implement one or more interfaces
abstract	Class with abstract methods OR abstract method declaration
interface	Pure contract — default methods allowed since Java 8
final	Variable: constant · Method: no override · Class: no extend
static	Belongs to class, not instance
this	Current object reference / call another constructor
super	Parent class reference / call parent constructor
instanceof	Runtime type check: if (a instanceof Dog d)
new	Allocate object on heap
synchronized	Mutual exclusion lock on method or block
volatile	Ensures visibility across threads (no caching)
transient	Exclude field from serialisation

13.2 OOP Rapid-Fire One-Liners

Question	Answer
Overloading vs Overriding	Overloading: same name, different params, compile-time. Overriding: same signature, subclass, runtime.
Abstract class vs Interface	Abstract: state + code + 1 parent. Interface: contract + no state + multiple allowed.
== vs equals	==: reference equality. equals(): content equality (if overridden).
Stack vs Heap	Stack: primitives + refs + frames, thread-local. Heap: objects, shared, GC.
String immutability	Strings cannot be modified. Any 'change' creates a new object.
Why no multiple inheritance Java	Diamond problem. Use interfaces instead.
Singleton thread-safe way	Bill Pugh (Holder class) or double-checked locking with volatile.
PECS rule	Producer Extends, Consumer Super — for generic wildcards.
ArrayList vs LinkedList	ArrayList: O(1) get. LinkedList: O(1) insert at head/tail.

HashMap internal working	Array of buckets + linked list/tree (Java 8+). hashCode() → bucket, equals() → exact key.
Checked vs Unchecked	Checked: must handle (IOException). Unchecked: optional (NPE, RuntimeException).
C++ virtual keyword	Required for runtime polymorphism. Java: all methods virtual by default.
C++ destructor rule	Always make base class destructor virtual if using polymorphism.
RAII	Constructor acquires, destructor releases — deterministic vs Java GC.

All the best! Crack that interview! 

OOP Complete Interview Preparation Guide — Java + C++